

A Holistic Approach to Reactive Mobile Manipulation

Jesse Haviland , Niko Sünderhauf , and Peter Corke 

Abstract—We present the design and implementation of a taskable reactive mobile manipulation system. In contrary to related work, we treat the arm and base degrees of freedom as a holistic structure which greatly improves the speed and fluidity of the resulting motion. At the core of this approach is a robust and reactive motion controller which can achieve a desired end-effector pose, while avoiding joint position and velocity limits, and ensuring the mobile manipulator is manoeuvrable throughout the trajectory. This can support sensor-based behaviours such as closed-loop visual grasping. As no planning is involved in our approach, the robot is never stationary *thinking* about what to do next. We show the versatility of our holistic motion controller by implementing a pick and place system using behaviour trees and demonstrate this task on a 9-degree-of-freedom mobile manipulator. Additionally, we provide an open-source implementation of our motion controller for both non-holonomic and omnidirectional mobile manipulators available at [jhavl.github.io/holistic](https://github.com/jhavl/holistic).

Index Terms—Mobile manipulation, motion control, kinematics, and reactive control.

I. INTRODUCTION

PROGRAMMING a robot to perform a task is a longstanding challenge in robotics and most systems employ some combination of planning and reactivity. Planning takes a current and desired world state and outputs a robot trajectory to effect that change. Solutions are optimal with respect to some measure but the plan is executed open-loop and its success is therefore critically reliant on the provided initial world state and the accuracy of the robot in following the plan. Planning systems today are extremely capable and can handle problems with many degrees of freedom and constraints, but the planning time can be a significant fraction of a second or more.

In contrast, reactive systems “close the loop” on sensory information. Such systems are highly responsive and robust to changes in their environment but typically do not handle complex constraints and the resulting behaviours are typically simplistic and not suitable for most real-world tasks.

Manuscript received September 9, 2021; accepted January 10, 2022. Date of publication January 27, 2022; date of current version February 4, 2022. This letter was recommended for publication by Associate Editor Y. Yang and Editor M. Vincze upon evaluation of the reviewers’ comments. This work was supported by the Queensland University of Technology (QUT) Centre for Robotics. (Corresponding author: Jesse Haviland.)

The authors are with the QUT Centre for Robotics, Brisbane, QLD 4000, Australia (e-mail: j.haviland@qut.edu.au; niko.suenderhauf@qut.edu.au; peter.corke@qut.edu.au).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2022.3146554>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2022.3146554

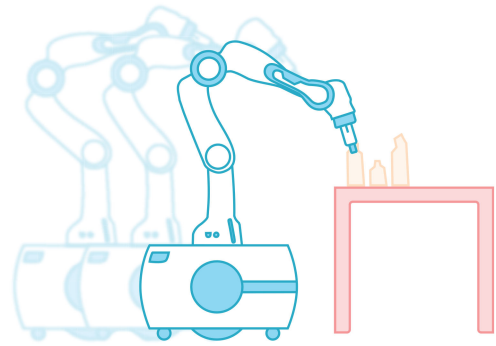


Fig. 1. Our reactive motion controller treats the mobile manipulator’s base and arm degrees of freedom as a holistic system. The resulting system can perform closed-loop visual grasping in dynamic environments while avoiding joint position and velocity limits and ensuring the mobile manipulator remains manoeuvrable throughout the motion.

The planning-reactivity dichotomy was identified long ago [1] and most robot systems today combine planning and reactive subsystems. For example, the global and local planner architecture is commonly used, and the “local planner” provides the system’s reactivity to the real world.

As displayed in Fig. 1, in this letter we extend our earlier work on reactive control for robot manipulator arms [2] to a mobile manipulator that includes a non-holonomic base. This provides the ability to command end-effector velocity in an infinitely large workspace with motion optimally partitioned between arm and base in a way that maximises arm dexterity and allows graceful and fluid motion of the whole mobile manipulator. We achieve holistic motion by considering all degrees-of-freedom simultaneously within a quadratic program.

However, there remains a significant gap between what a reactive behaviour can achieve and what is necessary to complete a useful task. A task is typically a sequence of behaviours which admits the need for detecting when a behaviour has completed, switching between behaviours according to world state and the goal, and error recovery.

To make our reactive system taskable we combine it with a behaviour tree. Compared to alternatives such as state machines, behaviour trees are easier to express and have inherent error recovery capability. The behaviour tree orchestrates the sensor-based behaviours of the reactive subsystem to achieve useful tasks. The behaviour tree can be considered as a form of scripting language, and still requires human ingenuity to create but could, in the future, be synthesised by a task-level planner. The result is a holistic system that can perform complex tasks

while exhibiting fluid and continuous motion without pauses for planning.

The contributions of this letter are:

- 1) a novel method of incorporating a non-holonomic mobile manipulation platform into a holistic reactive motion control framework with example code available at [jhavl.github.io/holistic](https://github.com/jhavl/holistic)
- 2) complete integration of a closed-loop mobile grasper using behaviour trees
- 3) reactive control and validation on a 9 ° of freedom non-holonomic mobile manipulation platform with a comparison to prior work.

II. RELATED WORK

Many mobile manipulation systems have been designed in recent years. Most prominently, the work in [3] used visual, tactile and proprioceptive sensor data to accomplish pick and place tasks using the PR2 robot. Graspable objects and support surfaces are segmented from 3D data, known objects have meshes fitted, and point clouds represent unknown objects. Grasp poses are achieved using an OMPL motion planner [4] which generates motion for the 7°-of-freedom arm to follow. The approach in [5] extends this to tightly couple visual and tactile sensors to enable grasp failure detection. The tactile feedback enables some local reactivity in grasping to account for sensor error or small grasp-target perturbances. Similar approaches are used on the Intel HERB platform in [6], [7]. Additionally, [8] uses a similar approach on the ARMAR-III robot for dual-arm grasping. In each of these works, the base of the robot is moved into position *before* actuating the arm to complete a grasp. While this simplifies the problem, the task is completed in a slow, and discontinuous manner where the motion is not *graceful*. Graceful motion is defined as being safe, comfortable, fast, and intuitive [9]. Specifically, the velocity must be smooth and consistent – not stop-start or unnatural.

Each of the above-mentioned works use motion planning to find an optimal trajectory subject to the supplied criteria. Motion planners can offer collision avoidance, joint limit avoidance, among many useful features. However, the associated computation time limits them to open-loop control. In a benchmark from [10], motion planners from the OMPL library range from 0.61 s to 1.3 s to compute a trajectory on a 7°-of-freedom arm, and between 18.7 s to 20.3 s to compute a trajectory on an 18 °-of-freedom PR2 robot. This look-then-move paradigm is slow and not robust to environmental change and sensor or motion error.

The NEO controller in [2] exploits the differential kinematics of a table-top manipulator to create a motion controller which can avoid both static and dynamic obstacles while also avoiding the joint position and velocity limits of the manipulator. The controller is purely reactive and is an extension of the classical resolved-rate motion control [11] which has been re-framed as a quadratic programming optimisation problem (QP). NEO's performance was shown to be comparable to motion planners for a reaching task with obstacles – while taking 9.8 ms to compute each step. An alternate approach to reactive motion control is

through potential fields where the robot is repelled from joint limits and obstacles while being attracted towards the goal [12], [13] and has been implemented on a mobile manipulator [14]. However, the potential field approach performs poorly compared to NEO [2] and becomes intractable for complex and cluttered environments. In [15], reactive whole-body control was achieved on a holonomic humanoid platform using a velocity controller on the mobile base and force control for the manipulators. The low latency of reactive motion controllers allows for greater versatility in the operation of the robot than planning based approaches, and makes tasks such as closed-loop visual grasping possible.

Reactive control of a non-holonomic mobile base is challenging since it cannot achieve an arbitrary task-space velocity through a time-invariant state-feedback controller [16]. This can be overcome with a switched control algorithm [17] or a dynamics-based approach [3]. The most prominent method of controlling a mobile base is with non-linear optimisation based motion planning [18].

Combining and incorporating various controllers into a mobile manipulation system can be just as difficult as designing the individual components [3]. An alternate control mechanism, Riemannian Motion Policies, fuses the outputs from different motion controllers to actuate the robot [19]. However, this concept is confined to motion generation and lacks the ability to coordinate perception, navigation and actuation. A common approach is to use finite-state machines (FSM) or similar architectures [5]. In recent times, behaviour trees have grown in popularity, and have been shown to be very useful for robotics where complex tasks need to be completed in a robust, reliable, and reactive manner [20]. Behaviour trees provide an alternative to FSMs and have several advantages such as readability, maintainability, and scalability, while also providing a greater degree of expressiveness [21]. The reactive nature of behaviour trees provides a better match for reactive motion controllers than FSMs as robot targets and state information can be updated quickly and easily based on visual/robot feedback without requiring very complicated and unreadable state flow. Additionally, these benefits extend to reactive error recovery which can be implemented simply and based on visual/robot feedback.

III. MODELLING A MOBILE MANIPULATOR

Mobile manipulators comprise a mobile base with one or more manipulators where the base is either omnidirectional or non-holonomic.

A. Two-Wheel Differential Drive Mobile Manipulators

The pose of the robot's end-effector is

$${}^0\mathbf{T}_e = {}^0\mathbf{T}_b(x, y, \theta) {}^b\mathbf{T}_a {}^a\mathbf{T}_e(\kappa_a, \mathbf{q}_a) \quad (1)$$

where $\{0\}$ is the world reference frame, ${}^0\mathbf{T}_b$ is the relative pose of the mobile base $\{b\}$, ${}^b\mathbf{T}_a$ is a constant relative pose from the mobile base frame to the base of the manipulator arm $\{a\}$, ${}^a\mathbf{T}_e$ is the forward kinematics of the arm where the end-effector

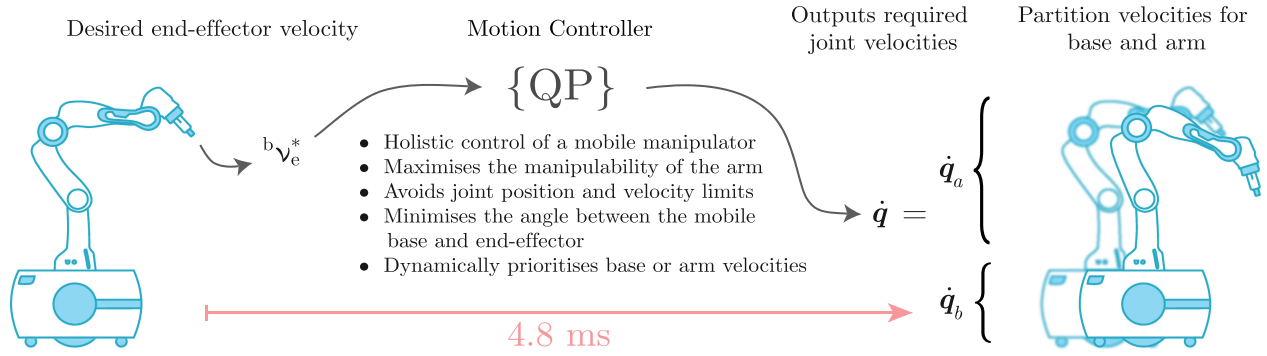


Fig. 2. Desired end-effector spatial velocity $^b\nu_e^*$ is taken as the input to our motion controller, which is expressed as a quadratic programme (QP). The motion controller minimises defined costs and incorporates constraints to ensure reliable, fast, and graceful control of the robot. The QP outputs joint velocities $\dot{q}$ to be actuated by the robot. We partition these joint velocities into the base virtual joint velocities $\dot{q}_b$ and arm joint velocities $\dot{q}_a$. The arm joint velocities are applied directly to the arm, while the base virtual joint velocities are transformed and applied to mobile base's wheels as described in Section III.

frame is $\{e\}$, κ_a are the arm kinematic parameters and q_a are the arm's joint coordinates.

Our approach is based on differential kinematics and we wish to determine a relationship between base velocity and joint angle rates $\dot{q}_a$, and the end-effector spatial velocity ν_e . To facilitate this we introduce an infinitesimal transform into the kinematic chain (1) to represent base instantaneous motion

$${}^0T_e = {}^0T_b \underbrace{{}^bT_{b'}(\delta_\theta, \delta_d)}_{\text{base instantaneous}} {}^{b'}T_a {}^aT_e(\kappa_a, q_a) \quad (2)$$

where δ_θ and δ_d are respectively infinitesimal rotation and forward translation of the non-holonomic base. We could consider this as a simple revolute-prismatic robot with virtual joint coordinates $q_b \in \mathbb{R}^2$, $\|q_b\| \rightarrow 0$ and $\dot{q}_{b,0} = \dot{\delta}_\theta$ and $\dot{q}_{b,1} = \dot{\delta}_d$. The resulting expression is now similar to the forward kinematics of a serial-link manipulator and using the approach of [22] we can write the differential kinematics as

$${}^0\nu_e = {}^0J_e(x, y, \theta, q) \begin{pmatrix} \dot{q}_b \\ \dot{q}_a \end{pmatrix} \quad (3)$$

which maps the velocity of all axes of the mobile manipulation system to the end-effector spatial velocity where $q = (0_2, q_a)^\top \in \mathbb{R}^n$ and n is the number of virtual and real joints. The holistic manipulator Jacobian ${}^0J_e(\cdot)$ is expressed in the world frame. The velocity of the virtual base joints are related to the wheel angular velocities by

$$\dot{q}_{b,0} = \frac{R}{W}(\varpi_R - \varpi_L), \dot{q}_{b,1} = \frac{R}{2}(\varpi_R + \varpi_L) \quad (4)$$

where R is the wheel radius, ϖ_R, ϖ_L are the robot's right and left wheel velocities respectively, and W is the distance between the two wheels.

For some control behaviours, it is useful to express velocity in different frames. For example, in the base frame ${}^b\nu_e$ or end-effector frame ${}^e\nu_e$ – the corresponding Jacobians ${}^bJ_e(q)$ and ${}^eJ_e(q)$ can be similarly derived.

Using the holistic manipulator Jacobian, we can construct a resolved-rate motion controller

$$\dot{q}^*(t) = {}^bJ_e(q)^+ {}^b\nu_e^*(t) \quad (5)$$

where $(\cdot)^+$ denotes the Moore-Penrose pseudoinverse, $\dot{q}^*$ is the joint velocities required for the mobile manipulator's end-effector to experience a desired spatial velocity and ${}^b\nu_e^* = (v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)^\top$. This controller represents the most basic form of reactive and holistic control of a mobile manipulator.

The demanded joint velocity $\dot{q}^*$ is partitioned into the base virtual joint velocities $\dot{q}_b^*$ which are transformed to the left and right wheel velocities using the unicycle kinematic model using (4), and $\dot{q}_a^*$ corresponds to the arm's joint velocities.

B. Omnidirectional Mobile Manipulators

Omnidirectional robots can instantaneously translate and rotate in the plane and using the virtual joint approach from above we define $q_b = (\delta_x, \delta_y, \delta_\theta) \in \mathbb{R}^3$.

IV. MOBILE MANIPULATOR MOTION CONTROLLER

We extend the NEO motion controller from [2] which was designed for table-top manipulators to enable holistic control of a mobile manipulator. We provide a high level overview of our approach in Fig. 2. This controller outputs joint velocities $\dot{q}$ that achieve a desired end-effector velocity ${}^b\nu_e^*$ minus a slack vector (intentional error to the desired velocity) δ where the slack component is used to allow the controller to better minimise costs and meet certain constraints. This controller is expressed as a QP

$$\begin{aligned} \min_x \quad & f_o(x) = \frac{1}{2}x^\top Qx + C^\top x, \\ \text{subject to} \quad & \mathcal{J}x = {}^b\nu_e, \\ & \mathcal{A}x \leq \mathcal{B} \\ & \mathcal{X}^- \leq x \leq \mathcal{X}^+ \end{aligned} \quad (6)$$

where $x = (q, \delta)^\top$ is the decision variable, Q incorporates joint velocity and slack costs, C incorporates manipulability maximisation and auxiliary performance tasks, $\mathcal{J}$ incorporates an augmented manipulator Jacobian, $\mathcal{A}$ and $\mathcal{B}$ implement joint position limit avoidance, and $\mathcal{X}^{+,-}$ limits maximum and minimum values of the decision variable. To lift the capabilities

of NEO [2] from stationary to mobile manipulators, we make significant changes to formulation of the QP which we detail below.

A. Velocity Control

The end-effector has a spatial velocity

$${}^b\boldsymbol{\nu}_e(t) = {}^b\boldsymbol{\nu}_e^*(t) - \boldsymbol{\delta}(t) \in \mathbb{R}^6 \quad (7)$$

where ${}^b\boldsymbol{\nu}_e^*(t) \in \mathbb{R}^6$ is the desired end-effector spatial velocity. $\boldsymbol{\delta}(t) \in \mathbb{R}^6$ is the slack velocity which allows for the end-effector to stray from the desired motion in order to satisfy the inequality constraints. This motion is created from the equality constraint in (6) with

$$\mathcal{J} = \begin{pmatrix} {}^b\mathbf{J}_e(\mathbf{q}) & \mathbf{1}_{6 \times 6} \end{pmatrix} \in \mathbb{R}^{6 \times (n+6)} \quad (8)$$

$$\mathbf{x} = \begin{pmatrix} \dot{\mathbf{q}} \\ \boldsymbol{\delta} \end{pmatrix} \in \mathbb{R}^{(n+6)} \quad (9)$$

where $\mathbf{1}$ is the identity matrix.

B. The Objective Function

The optimisation will minimise $\mathbf{x}$ in the objective function subject to the equality and inequality constraints. The objective function in (6) seeks to minimise the joint velocity norm, minimise the slack norm, and maximise the manipulability of the robot according to the measure from [23]. More information on manipulability maximisation is available in [2], [24], [25]. The objective function is defined by

$$\mathcal{Q} = \begin{pmatrix} \text{diag}(\boldsymbol{\lambda}_q) & \mathbf{0}_{6 \times 6} \\ \mathbf{0}_{n \times n} & \text{diag}(\boldsymbol{\lambda}_\delta) \end{pmatrix} \in \mathbb{R}^{(n+6) \times (n+6)}, \quad (10)$$

$$\mathcal{C} = \begin{pmatrix} \hat{\mathbf{J}}_m + \boldsymbol{\epsilon} \\ \mathbf{0}_{6 \times 1} \end{pmatrix} \in \mathbb{R}^{(n+6)} \quad (11)$$

where $\boldsymbol{\lambda}_q \in \mathbb{R}^n$ adjusts between minimising the joint velocity norm and maximising manipulability, $\boldsymbol{\lambda}_\delta \in \mathbb{R}^n$ adjusts the cost of adding slack, $\hat{\mathbf{J}}_m$ is the manipulability Jacobian, and $\boldsymbol{\epsilon}$ is the base to end-effector angle.

Manipulability $\hat{\mathbf{J}}_m$: The manipulability metric $m(\mathbf{q})$ describes how easily a manipulator can achieve an arbitrary end-effector velocity. The base is capable of providing 2 (or 3 for a holonomic platform) degrees-of-freedom to contribute towards the end-effector velocity with an infinite workspace. In contrast, the arm will typically have more degrees-of-freedom but a finite workspace. To ensure the arm is capable of exploiting the extra degrees-of-freedom to provide any arbitrary end-effector velocity at a given time, we calculate the manipulability Jacobian using the arm joints

$$\hat{\mathbf{J}}_m = \begin{pmatrix} \mathbf{0}_b, & \mathbf{J}_m^a(\mathbf{q}_a) \end{pmatrix} \quad (12)$$

before being used in (11), where $\mathbf{0}_b$ is a zero vector, and $\mathbf{J}_m^a(\mathbf{q}_a)$ is the manipulability Jacobian corresponding to arm joints [2].

Base Orientation θ_ϵ : Fig. 3 shows the base to end-effector angle θ_ϵ . When $\theta_\epsilon \simeq 0^\circ$ the platform has infinite reach in the

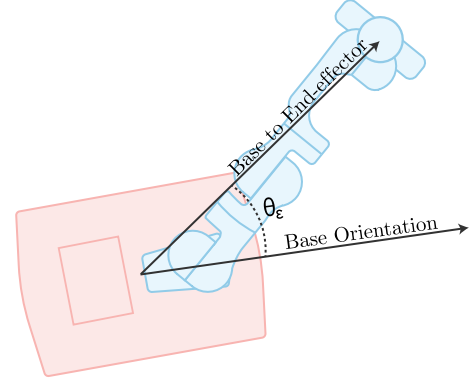


Fig. 3. Motion controller seeks to minimise the angle θ_ϵ between the base orientation and the end-effector which ensures that the mobile manipulator remains highly manoeuvrable.

forward direction but when $\theta_\epsilon \simeq \pm 90^\circ$, the platform's reach is limited to that of the arm. In our approach, the end-effector leads towards the goal, therefore, maximum reach is achieved when $\theta_\epsilon \simeq 0^\circ$. The linear component of the cost function causes the optimiser to choose joint velocities which orient the mobile base towards the end-effector as described by Fig. 3. This cost is

$$\boldsymbol{\epsilon} = \begin{pmatrix} -k_\epsilon \theta_\epsilon, & \mathbf{0}_{n-1} \end{pmatrix} \quad (13)$$

where $\theta_\epsilon = \text{atan2}({}^b\mathbf{T}_{e_{1,3}}, {}^b\mathbf{T}_{e_{0,3}})$ and k_ϵ is a gain which adjusts how aggressively the base will be re-oriented.

Slack Cost λ_δ : The slack cost determines the size of the slack vector in the output of the optimiser. A large value limits the slack possible and associated benefits, while a small value leads to the possibility that the slack will cancel out the desired velocity, leaving a large steady-state error. We choose $\lambda_\delta = \frac{1}{\|e\|}$ where $\|e\|$ is the Euclidean distance error between the end-effector and the desired end-effector pose. This provides the optimiser the freedom to avoid operational limits and maximise manipulability at the beginning of the trajectory, while the increasing restriction ensures the end-effector continues to approach the goal.

Joint Velocity Cost λ_q : Robot arms have a finite reach and a limited joint range in which they are highly manipulable, so we choose the joint velocity cost in the QP

$$\boldsymbol{\lambda}_q = \begin{pmatrix} \frac{1}{\|e\|_b}, & k_a \end{pmatrix} \quad (14)$$

where the variable gain applies to the mobile base joints, and the constant k_a is applied to the arm joints. This causes the optimiser to favour the mobile-base degrees of freedom when the error $\|e\|$ is large, and the arm degrees of freedom when the error is small.

C. Joint Position Limit Avoidance

Joint position-limit avoidance is implemented using velocity dampers. The velocity damper will restrict joint velocities of the robot to reduce or nullify the rate at which it is approaching a joint limit. Note that we do not perform joint position limit avoidance on the virtual base joints. The joint position-limit

avoidance velocity damper is defined as

$$\mathcal{A} = (\mathbf{1}_{n \times n+6}) \in \mathbb{R}^{(n \times n+6)} \quad (15)$$

$$\mathcal{B} = \begin{pmatrix} \mathbf{0}_b \\ \eta \frac{\rho_0 - \rho_s}{\rho_i - \rho_s} \\ \vdots \\ \eta \frac{\rho_n - \rho_s}{\rho_i - \rho_s} \end{pmatrix} \in \mathbb{R}^n \quad (16)$$

where ρ is the distance to the nearest joint limit, ρ_i is the influence distance in which to operate the damper, and ρ_s is the minimum distance for a joint to its limit.

D. Position-Based Servoing

We use the motion controller within a position-based servoing (PBS) scheme. This controller seeks to drive the robot's end-effector from its current pose to a desired pose. PBS is formulated as

$${}^b\boldsymbol{\nu}_e^* = \beta \psi \left(({}^b\mathbf{T}_e)^{-1} {}^b\mathbf{T}_{e^*} \right) \quad (17)$$

where $\beta \in \mathbb{R}^+$ is a gain term, ${}^b\mathbf{T}_{e^*} \in \mathbf{SE}(3)$ is the desired end-effector pose in the robot's base frame, and $\psi(\cdot) : \mathbb{R}^{4 \times 4} \mapsto \mathbb{R}^6$ is a function which converts a homogeneous transformation matrix to a spatial displacement.

V. EXPERIMENTS

We first validate our motion controller by demonstrating how a mobile manipulator travels between various poses, before demonstrating our proposed architecture performing a mobile pick and place behaviour task.

A. Platform Description

We use our in-house developed mobile manipulation platform Frankie for the following experiments. Frankie comprises an Omron LD-60 two-wheel differential-drive base and a 7°-of-freedom Franka-Emika Panda manipulator. In Experiment 1 and 2 we additionally use an Omni-Frankie model in simulation which has a holonomic base (which could be realised using mecanum wheels) to exhibit holistic operation with a holonomic mobile manipulator. The models of Frankie and Omni-Frankie are provided as part of the Robotics Toolbox for Python [26]. For real-world experiments, we use an Intel RealSense D435 RGB-D camera attached to the end-effector for visual feedback to support visual grasping. The Omron LD-60 mobile base performs localisation and mapping on an embedded computer using an in-built LiDAR. We control the platform using the software package from github.com/qcr/ros_omron_agv. Frankie contains an Intel i5 NUC and an Nvidia Xavier to perform image processing.

B. Experiment 1: Holistic Motion Controller Validation

In this simulated experiment, we first show how our proposed motion controller responds to various desired end-effector poses. Starting from the same initial pose, the base and arm must move to achieve a desired end-effector pose

- 4 m in front of the robot.
- 4 m to the right of the robot.
- 4 m behind the robot.
- 4 m in front of the robot, where the target moves forwards and to the left for 6 s and then to the right for 6 s before becoming stationary.

The distance 4 m was chosen as it is sufficient to require base and arm motion.

C. Experiment 2: Random Reaching Task

In this simulated random reaching task, the mobile manipulator must reach 1000 randomly generated valid and reachable end-effector poses, located within a 4 m radius of $\{0\}$. Using this methodology, we:

- compare the success rate, on both Frankie and Omni-Frankie, of our Holistic controller with separate base and arm planners (using RRTConnect algorithm from OMPL [4]) which represents the approach from [3].
- compare the success rate of Frankie using our holistic controller while adjusting k_e introduced in Section IV.
- compare the success rate of Frankie using our holistic controller while adjusting $\hat{\mathbf{J}}_m$ introduced in Section IV.

D. Experiment 3: Repetitive Pick and Place Task

We evaluate the merits of our approach in a real-world repetitive pick-and-place task. In this experiment, the mobile manipulator must pick common household objects from a container and place them on a table located 3 m across the room from the original container.

To make our system taskable and responsive to many competing goals and states, we use behaviour trees. A behaviour tree is a directed acyclic graph with a single root vertex. Vertices represent behaviours and edges represent the flow of control. Vertex execution is governed by ticks which are an activation signal. The ticks are propagated through the tree at a defined frequency. Upon receiving a tick, the vertex returns running if it is still executing, success if its goal was achieved, or otherwise failure. This process makes behaviour trees highly reactive, as the whole tree gets processed on every tick. A comprehensive description of behaviour trees can be found in [20].

We have implemented a behaviour tree to complete this pick-and-place task. We assume the robot is localised within a two-dimensional map of the environment, and the container/table locations within this map are known. We use GG-CNN to perform grasp point synthesis [27] and a RealSense D435 eye-in-hand camera to provide depth images. GG-CNN consumes depth images and outputs the best grasp pose visible from the depth image.

The behaviour tree will first run a branch which recovers the arm from any errors (encountered by joint position, velocity, force limits etc.) and moves the arm into a ready pose and the base to the initial pose. Any unhandled failure will cause the tree to return to this branch where errors are recovered and the platform is re-initialised. The second branch of the tree will repeat on success. This means that the pick and place task will repeat unless there is a failure.

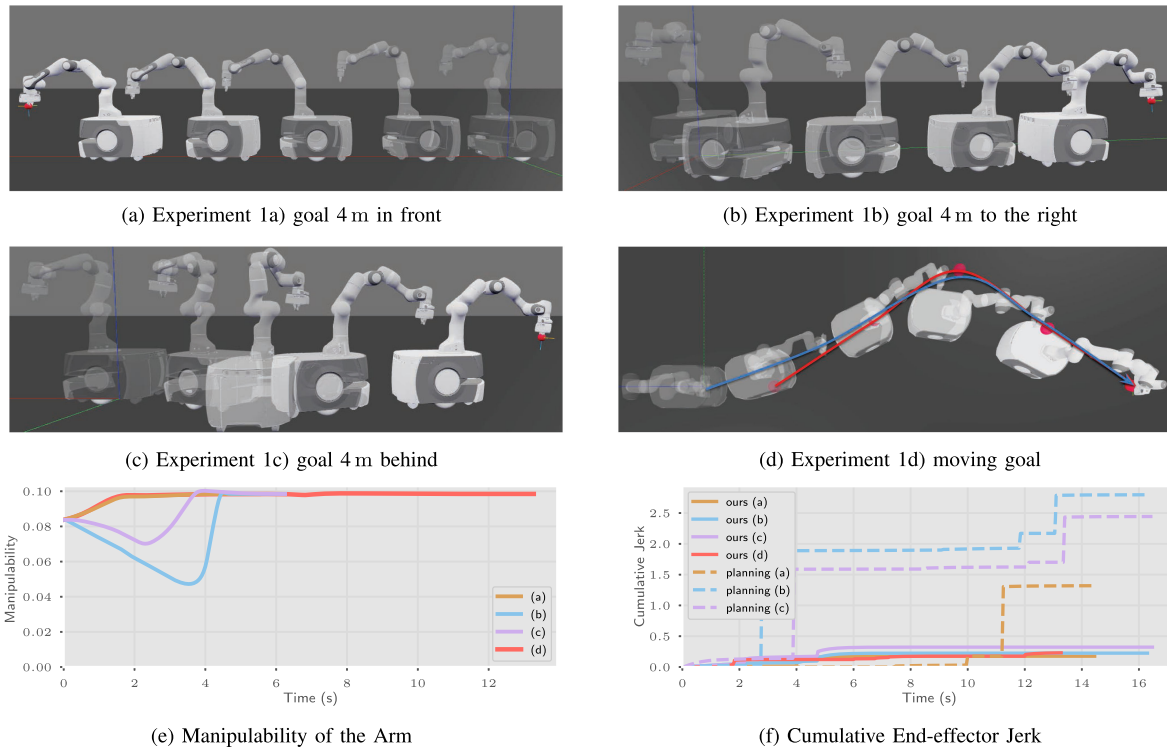


Fig. 4. Frankie mobile manipulator moving from a starting pose to a desired pose using our proposed motion controller where robot opacity increases with time. (a) Goal is directly in front of the robot. (b) Goal is to the side of the robot. (c) Goal is directly behind the robot. (d) The goal is moving for 12 s (red line) and the end-effector (blue line) catches and tracks it. (e) The manipulability of the arm throughout (a)–(d). (f) Cumulative end-effector jerk comparing the holistic and planning based approaches.

In the second branch, the robot will first attempt to grasp an object. The grippers are opened if required, and then the platform moves to a defined pickup pose using our Holistic Motion Controller from which the container holding the objects is visible to the eye-in-hand camera. Subsequently, a grasp is attempted. In a grasp attempt, the Holistic Motion Controller guides the robot towards the grasp pose output from GG-CNN. When the distance to the desired grasp pose is greater than 0.25 m, the output from GG-CNN is continuously updating the Holistic Motion Controller goal. Due to the minimum sensing distance of the RealSense D-435 camera, the grasp pose is locked in when it is 0.25 m away from the camera. After a grasp attempt, the arm retreats to a ready configuration where grasp success is detected. A failure is detected by the gripper being fully closed (i.e. the hand is empty), or the arm encounters an error. If a failure is detected, the grasp attempt is repeated.

Upon a successful grasp, the platform moves to a dropoff pose where the object is placed. The pickup and dropoff locations are separated by approximately 3 m. The pick and place sequence is then repeated.

To perform a comparison to prior work, we repeated the pick and place task while using motion planners instead of our holistic controller. In this implementation, we use the internal Omron planner to control the mobile base, and the RRTConnect motion planner from OMPL to control the arm [4]. This mimics the robot control approach from [3]. The behaviour tree and vision stack remain the same, except we only use the first output from GG-CNN due to the open-loop nature of motion planners.

VI. RESULTS

The average execution time of the holistic motion controller during the experiments was 4.8 ms using an Intel i5 NUC with 4 cores at 2.3 GHz in a single threaded process – a control rate just above 200 Hz. We use the following values for the controller parameters: $\eta = 1$, $\rho_i = 50^\circ$, and $\rho_s = 2^\circ$ in (16), $k_e = 0.5$ in (13), $k_a = 0.01$ in (14), and $\beta = 1$ in (17). The effects of changing the controller parameters is discussed in Experiment 2 results. In practice we found that performance is sound within 50% to 200% of the chosen gain values.

A. Experiment 1 Results

We display the robot motion of Frankie from Experiment 1 in Fig. 4 using the Swift Simulator [26], however we refer readers to the supplementary video which shows the robot motion with a comparison to the non-holistic planning-based approach, and the motion of the holonomic Omni-Frankie robot. We also provide code examples of our motion controller running on the Frankie and Omni-Frankie mobile manipulators at jhavl.github.io/holistic.

As shown by Fig. 4, the mobile manipulator achieves all four poses set out by the experiment. These motions took 5.42, 6.17, 6.17, and 13.12 seconds to complete and correspond to Fig. 4(a), (b), (c), and (e) respectively. We repeated Experiment 1 using a method representative of the non-holistic planning-based approach from [3]. Using this approach, the robot took 14.32, 16.17, and 16.45 seconds to complete the motion respectively.

TABLE I
EXPERIMENT 2A) COMPARING NUMBER OF FAILURES OVER 1000 RANDOM REACHING TASKS

Frankie (non-holonomic)		Omni-Frankie (holonomic)	
Motion Planning	Holistic (ours)	Motion Planning	Holistic (ours)
71	35	71	36

TABLE II
EXPERIMENT 2B) EFFECT OF k_ϵ ON PERFORMANCE OVER 1000 RANDOM REACHING TASKS

k_ϵ	0.0	0.1	0.5	1.0
Failures	38	34	39	76
Mean Final θ_ϵ	22.91°	10.31°	4.58°	3.43°

which is between two and three times slower than the holistic reactive approach. Note that there is no planning-based approach for Experiment 1 d) where the goal moves, due to the open-loop nature of planners.

Fig. 4(a) shows the simplest of the motions, and the base and arm cooperate so the arm remains well-conditioned and not overly stretched out. Fig. 4(b) shows a slightly more complicated motion as the base must rotate and translate in cooperation with the arm to reach the goal. The base rotates in cooperation with the arm motion preventing the arm from approaching its joint limits. Fig. 4(c) shows a very complex task requiring close cooperation between base and arm. The base rotates and translates to ensure the arm does not encounter joint limits in the motion. Fig. 4(d) shows that our controller can instantaneously, and smoothly react to a changing goal pose. Notably, these motions are not the result of a computed trajectory, it is an emergent behaviour of the reactive holistic motion controller. Fig. 4(e) shows that in all four motions, the robot arm remains well-conditioned throughout when using our proposed controller.

Fig. 4(f) shows the cumulative end-effector jerk in each motion when using our proposed holistic controller, and the non-holistic planning based approach where the jerk from the initial robot motion has been ignored. Lower cumulative jerk indicates smoother overall motion and we see that the holistic approach generates quantitatively smoother motion than previous work which involves disjointed stop-start motion.

B. Experiment 2 Results

Table I shows that for both holonomic and non-holonomic mobile manipulators, our holistic controller outperforms the non-holistic motion planners. Failures in the holistic controller are caused by local minima, while motion planning failures are the result of a solution not being found within 5 s.

Table II shows the effect of k_ϵ (13) on performance. To maximise the ability for the robot to complete follow-on tasks, including grasping, it is important to minimise the base to end-effector angle θ_ϵ . Making k_ϵ large will minimise θ_ϵ but leads to more controller failures due to ill-conditioned arm configurations leading to local minima. Table II shows that $k_\epsilon = 0.1 \sim 0.5$ provides suitable results.

TABLE III
EXPERIMENT 2C) EFFECT OF $\hat{\mathbf{J}}_m$ ON PERFORMANCE OVER 1000 RANDOM REACHING TASKS

$\hat{\mathbf{J}}_m$	$(\mathbf{0}_b, \mathbf{0}_a)$	$\mathbf{J}_m^{ba}(\mathbf{q})$	$(\mathbf{0}_b, \mathbf{J}_m^a(\mathbf{q}_a))$
Failures	161	161	39
Mean Final $m(\mathbf{q}_a)$	0.032	0.033	0.079

TABLE IV
PICK AND PLACE TIMING COMPARISON

Approach	Mean Grasp Time	Mean Pick and Place Time
ours	5.2 s	16.8 s
planning	15.6 s	35.2 s
[3]	15.4 s	-
[7]	30 s	-
[6]	-	51 s

Table III shows the effect which $\hat{\mathbf{J}}_m$ has in (12) on performance. These results show that low arm manipulability $m(\mathbf{q}_a)$ leads to operational failures which is consistent with [2]. When manipulability is not optimised with $\hat{\mathbf{J}}_m = (\mathbf{0}_b, \mathbf{0}_a)$ and when manipulability of the whole platform is optimised with $\mathbf{J}_m^{ba}(\mathbf{q})$ the average final arm manipulability is low which leads to increased failures. Therefore, it is important to maximise the arm's ability to exploit its limited workspace.

C. Experiment 3 Results

We repeated Experiment 3 ten times for a total of 100 objects picked and placed. The robot successfully picked and placed all 100 objects with 113 grasp attempts. We obtained the same grasp performance when we repeated the task while using the motion planning approach instead of our holistic controller. In this experiment, there were no failures caused by the motion control or robot actuation. Minor failures were caused by the vision pipeline leading to incorrect grasp points from the GG-CNN network. Failed grasp attempts were caught by the behaviour tree and caused the grasp to be re-attempted. A full run of Experiment 3 is shown in our supplementary video.

For all robotic applications, task performance time is an important performance metric. While we measure and report our motion times in Table IV, we were not able to find timing results in other letters reporting similar robotic tasks. For example, in [3] the robot picks an object from a bench-top containing several household objects, while in [7], the robot picks an object from a bench-top containing a single household object. In [6], a mobile manipulator must pick and place cups from a bench where no base motion is required. The task and movement requirements are identical to our task, or easier in the case of [6], so it is interesting to compare motion times for various approaches. It would be preferable to use published timing results but, in its absence, we have reverse engineered these from analysis of published supplementary video material and it is summarized in Table IV. We could encourage future authors to include time and distance data in their letters to encourage meaningful comparisons.

The timing results from Experiment 3, using motion planners, is also shown in Table IV under the *planning* approach. Grasp time is defined as the time spent in the grasp motion, which results in the object being grasped (including repeat grasp attempts where applicable), and pick and place time is defined as grasp time plus travelling to the drop-off point and placing the object. The planning based approaches all offered good grasp success rates but took much longer to complete the task. As shown in Table IV, our mean grasp time is 3 times faster than the motion planning approach and [3] while being 6 times faster than [7]. Our average pick and place time is 2 times faster than the motion planning approach and 3 times faster than [6]. These timings are consistent with the timing comparison in Experiment 1. Results from these experiments suggest that our holistic and reactive motion controller can significantly improve task execution time on a mobile manipulator while also being reliable and robust to task completion. The resulting motion is graceful: intuitive, smooth and fast [9], while the time spent with the robot stationary has been completely eliminated.

VII. CONCLUSION

In this letter, we proposed a novel motion controller which provides reactive and holistic control of both non-holonomic and holonomic mobile manipulation platforms. Our experiments demonstrate that our purely reactive and holistic approach performs faster, more efficiently, and more robustly than related work. In combination with a behaviour tree, our reactive system becomes taskable and capable of error recovery as demonstrated by a mobile pick and place task where the motion is smooth, intuitive and fast.

Thanks to the versatility of behaviour trees, our behaviour tree can be readily modified to perform different tasks, or extended with [21] to automatically generate behaviour trees based on natural language input. In the future, we would like to implement perception related behaviours, such as object recognition and collision detection, which makes a wider range of tasks solvable. Additionally, with more perception we can incorporate the reactive collision avoidance from [2] into the holistic motion controller.

We have provided full examples of our motion controller running on the Frankie and Omni-Frankie mobile manipulators at [jhavl.github.io/holistic](https://github.com/jhavl/holistic).

REFERENCES

- [1] J. Albus, R. Quintero, and R. Lumia, "An overview of nasrem-the NASA/NBS standard reference model for telerobot control system architecture," in *Proc. Space Programs Technol. Conf. Exhibit*, 1994, Art. no. 4608.
- [2] J. Haviland and P. Corke, "NEO: A novel expeditious optimisation algorithm for reactive motion control of manipulators," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1043–1050, Jan. 2021.
- [3] S. Chitta, E. G. Jones, M. Ciocarlie, and K. Hsiao, "Mobile manipulation in unstructured environments: Perception, planning, and execution," *IEEE Robot. Automat. Mag.*, vol. 19, no. 2, pp. 58–71, Jun. 2012.
- [4] I. A. Sucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Automat. Mag.*, vol. 19, no. 4, pp. 72–82, Dec. 2012.
- [5] M. Ciocarlie, K. Hsiao, E. G. Jones, S. Chitta, R. B. Rusu, and I. A. Sucan, "Towards Reliable Grasping and Manipulation in Household Environments," in *Experimental Robotics*, Berlin, Germany: Springer, 2014, pp. 241–252.
- [6] S. Srinivasa *et al.*, "The robotic busboy: Steps towards developing a mobile robotic home assistant," *Intell. Auton. Syst.*, 10, 2008, pp. 129–136.
- [7] S. S. Srinivasa *et al.*, "Herb: A home exploring robotic butler," *Auton. Robots*, vol. 28, no. 1, pp. 5–20, 2010.
- [8] N. Vahrenkamp, M. Do, T. Asfour, and R. Dillmann, "Integrated grasp and motion planning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2010, pp. 2883–2888.
- [9] S. Gulati and B. Kuipers, "High performance control for graceful motion of an intelligent wheelchair," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2008, pp. 3932–3938.
- [10] J. Schulman, J. Ho, A. X. Lee, I. Awwal, H. Bradlow, and P. Abbeel, "Finding locally optimal, collision-free trajectories with sequential convex optimization," in *Proc. Robot.: Sci. Syst.*, 2013, vol. 9, pp. 1–10.
- [11] D. E. Whitney, "Resolved motion rate control of manipulators and human prostheses," *IEEE Trans. Man-Mach. Syst.*, vol. 10, no. 2, pp. 47–53, Jun. 1969.
- [12] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," in *Autonomous Robot Vehicles*, Berlin, Germany: Springer, 1986, pp. 396–404.
- [13] D.-H. Park, H. Hoffmann, P. Pastor, and S. Schaal, "Movement reproduction and obstacle avoidance with dynamic movement primitives and potential fields," in *Proc. 8th IEEE-RAS Int. Conf. Humanoid Robots*, 2008, pp. 91–98.
- [14] J. M. Cameron, D. C. MacKenzie, K. R. Ward, R. C. Arkin, and W. J. Book, "Reactive control for mobile manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 1993, pp. 228–235.
- [15] A. Dietrich, T. Wimbock, A. Albu-Schaffer, and G. Hirzinger, "Reactive whole-body control: Dynamic mobile manipulation using a large number of actuated degrees of freedom," *IEEE Robot. Automat. Mag.*, vol. 19, no. 2, pp. 20–33, Jun. 2012.
- [16] A. Bloch and B. Brogliato, "Nonholonomic mechanics and control," *Appl. Mech. Rev.*, vol. 57, no. 1, pp. 235–313, 2004.
- [17] V. Sankaranarayanan and A. D. Mahindrakar, "Switched control of a nonholonomic mobile robot," *Commun. Nonlinear Sci. Numer. Simul.*, vol. 14, no. 5, pp. 2319–2327, 2009.
- [18] J.-C. Latombe, *Robot Motion Planning*, vol. 124, Springer Science & Business Media, Boston, MA, USA, 2012.
- [19] N. D. Ratliff, J. Issac, D. Kappler, S. Birchfield, and D. Fox, "Riemannian motion policies," 2018, *arXiv:1801.02854*.
- [20] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*, Boca Raton, FL, USA: CRC Press, 2018.
- [21] G. Suddrey, B. Talbot, and F. Maire, "Learning and executing re-usable behaviour trees from natural language instruction," 2021, *arXiv:2106.01650*.
- [22] J. Haviland and P. Corke, "A systematic approach to computing the manipulator jacobian and hessian using the elementary transform sequence," 2020, *arXiv:2010.08696*.
- [23] T. Yoshikawa, "Manipulability of robotic mechanisms," *Int. J. Robot. Res.*, vol. 4, no. 2, pp. 3–9, 1985.
- [24] J. Haviland and P. Corke, "A purely-reactive manipulability-maximising motion controller," 2020, *arXiv:2002.11901*.
- [25] S. Patel and T. Sobh, "Manipulator performance measures-a comprehensive literature survey," *J. Intell. Robot. Syst.*, vol. 77, no. 3-4, pp. 547–570, 2015.
- [26] P. Corke and J. Haviland, "Not your grandmother's toolbox - the robotics toolbox reinvented for python," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 11357–11363.
- [27] D. Morrison, J. Leitner, and P. Corke, "Closing the loop for robotic grasping: A real-time, generative grasp synthesis approach," in *Proc. Robot.: Sci. Syst. XIV*, Jun. 2018, pp. 1–10.